

Compte-rendu (Google Skills / GCP) : GSP1185 – Securing Container Builds

Artifact Registry : Standard, Remote et Virtual Repositories

1 Source et objectif du lab

Ce document résume le lab **GSP1185** (Securing Container Builds) et explique, **commande par commande**, ce que fait chaque instruction utilisée dans Cloud Shell. La source est le texte de la page de lab fourni.

Objectifs annoncés :

- Créer et utiliser des **Standard Repositories** pour des packages privés (Maven).
- Créer un **Remote Repository** pour **caler Maven Central**.
- Créer un **Virtual Repository** qui agrège plusieurs dépôts via une politique d'upstream.

2 Pré-requis (contexte)

Le lab se fait dans **Google Cloud Shell** (VM prête à l'emploi) avec **gcloud** déjà installé et l'authentification déjà configurée via le compte temporaire du lab.

3 Commandes et explications

(Optionnel) Vérifier l'identité (authentification gcloud)

```
gcloud auth list
```

- Liste les comptes Google connus de l'outil **gcloud**.
- Indique lequel est **actif** (marqué par *).
- Utile pour vérifier que vous utilisez bien le compte temporaire du lab.

Commande suggérée (dans la sortie) : changer le compte actif

```
gcloud config set account 'ACCOUNT'
```

- Définit le compte actif utilisé par défaut par **gcloud**.
- Ici 'ACCOUNT' représente l'email/compte à sélectionner.

(Optionnel) Vérifier le projet courant

```
gcloud config list project
```

- Affiche la configuration courante de **gcloud** liée au projet.
- Permet de confirmer que **gcloud** pointe vers le bon **PROJECT_ID**.

3.1 Définir des variables d'environnement projet

```
export PROJECT_ID=$(gcloud config get-value project)
export PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --format='value(
projectNumber)')
```

- `PROJECT_ID` : récupère l'identifiant du projet configuré dans `gcloud`.
- `PROJECT_NUMBER` : récupère le **numéro** du projet (identifiant numérique interne GCP).
- `export` rend ces variables disponibles pour les commandes suivantes dans la session shell.

3.2 Activer l'API Artifact Registry

```
gcloud services enable artifactregistry.googleapis.com
```

- Active l'API **Artifact Registry** sur le projet.
- Sans cette activation, les commandes `gcloud artifacts ...` échoueront.

3.3 Récupérer des exemples (code du lab) et se placer dans le bon dossier

```
git clone https://github.com/GoogleCloudPlatform/java-docs-samples
cd java-docs-samples/container-registry/container-analysis
```

- `git clone` télécharge le dépôt d'exemples Java officiels Google Cloud.
- `cd` change le répertoire courant vers l'exemple utilisé par le lab.

4 Task 1 – Standard Repository (packages privés Maven)

4.1 Créer un dépôt Maven standard

```
gcloud artifacts repositories create container-dev-java-repo \
  --repository-format=maven \
  --location=us-central1 \
  --description="Java package repository for Container Dev Workshop"
```

- Crée un dépôt Artifact Registry nommé `container-dev-java-repo`.
- `-repository-format=maven` : le dépôt accepte des artefacts Maven (JAR, POM, etc.).
- `-location=us-central1` : hébergement régional (latence / conformité).
- Mode implicite : **STANDARD_REPOSITORY** (stockage de vos artefacts privés).

4.2 Décrire le dépôt

```
gcloud artifacts repositories describe container-dev-java-repo \
  --location=us-central1
```

- Affiche les métadonnées du dépôt : format, mode, taille, timestamps, chiffrement, etc.
- Permet de vérifier que la création a réussi et que le dépôt est vide au départ.

5 Task 2 – Configurer Maven pour Artifact Registry

5.1 Générer la configuration Maven (XML) à insérer dans le projet

```
gcloud artifacts print-settings mvn \  
  --repository=container-dev-java-repo \  
  --location=us-central1
```

- Produit des extraits XML pour Maven (typiquement `<distributionManagement>`, `<repositories>` et une `extension`).
- Objectif : dire à Maven
 - où **publier** les artefacts (`distributionManagement`),
 - où **télécharger** des dépendances (`repositories`),
 - comment s'authentifier/transport (`artifactregistry-maven-wagon`).

5.2 Ouvrir l'éditeur Cloud Shell

```
cloudshell workspace .
```

- Ouvre l'espace de travail/éditeur intégré de Cloud Shell dans le répertoire courant.
- Sert à modifier `pom.xml` pour y coller les sections XML générées.

5.3 Déployer (publier) le package Java dans Artifact Registry

```
mvn deploy -DskipTests
```

- Lance la phase Maven `deploy` : compilation + packaging + publication dans le dépôt défini par `distributionManagement`.
- `-DskipTests` : évite l'exécution des tests pour accélérer le déploiement.
- Résultat attendu : un artefact (ex. `hello-world`) apparaît dans `container-dev-java-repo`.

6 Task 3 – Remote Repository (cache Maven Central)

6.1 Créer un dépôt distant (remote) pointant vers Maven Central

```
gcloud artifacts repositories create maven-central-cache \  
  --project=$PROJECT_ID \  
  --repository-format=maven \  
  --location=us-central1 \  
  --description="Remote repository for Maven Central caching" \  
  --mode=remote-repository \  
  --remote-repo-config-desc="Maven Central" \  
  --remote-mvn-repo=MAVEN-CENTRAL
```

- Crée un dépôt en mode **REMOTE_REPOSITORY**.
- Ce dépôt agit comme un **proxy/cache** : quand Maven demande une dépendance de Maven Central, Artifact Registry la récupère et la **met en cache** dans ce dépôt.
- Intérêts : fiabilité, contrôle, sécurité (réduction de risques liés aux dépendances externes).

6.2 Inspecter le dépôt remote

```
gcloud artifacts repositories describe maven-central-cache \
  --location=us-central1
```

- Vérifie le mode (`remote-repository`), la cible (Maven Central) et la configuration.

6.3 Générer la config Maven pour utiliser le cache

```
gcloud artifacts print-settings mvn \
  --repository=maven-central-cache \
  --location=us-central1
```

- Fournit la section `<repository>` à ajouter au `pom.xml` pour que Maven aille chercher les dépendances via Artifact Registry.
- On change souvent l'id en `central` pour éviter un doublon d'identifiant dans `pom.xml`.

6.4 Créer une extension Maven “core” (pour résoudre parents/plugins tôt)

```
mkdir .mvn
cat > .mvn/extensions.xml << EOF
<extensions xmlns="http://maven.apache.org/EXTENSIONS/1.0.0" xmlns:xsi="http://www.w3.org
  /2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/EXTENSIONS/1.0.0 http://maven.apache.org/xsd
    /core-extensions-1.0.0.xsd">
  <extension>
    <groupId>com.google.cloud.artifactregistry</groupId>
    <artifactId>artifactregistry-maven-wagon</artifactId>
    <version>2.2.0</version>
  </extension>
</extensions>
EOF
```

- `mkdir .mvn` : crée le dossier de configuration Maven du projet.
- `extensions.xml` : déclare une extension **chargée très tôt** par Maven (avant `pom.xml`).
- L'extension `artifactregistry-maven-wagon` apporte la couche transport/auth nécessaire pour accéder à Artifact Registry, y compris pour des **parents** ou **plugins** Maven.

6.5 Forcer une reconstruction propre et compiler via le Remote Repository

```
rm -rf ~/.m2/repository
mvn compile
```

- `rm -rf ~/.m2/repository` supprime le cache local Maven (dépendances déjà téléchargées).
- `mvn compile` re-télécharge les dépendances nécessaires *via* Artifact Registry.
- Effet attendu : le dépôt remote (`maven-central-cache`) se remplit d'artefacts mis en cache.

7 Task 4 – Virtual Repository (agrégation via upstream policy)

7.1 Créer un fichier de politique d'upstream (priorités)

```
cat > ./policy.json << EOF
[
  {
    "id": "private",
    "repository": "projects/${PROJECT_ID}/locations/us-central1/repositories/container-dev-
      -java-repo",
    "priority": 100
  },
  {
    "id": "central",
    "repository": "projects/${PROJECT_ID}/locations/us-central1/repositories/maven-central-
      -cache",
    "priority": 80
  }
]
EOF
```

- Crée un fichier JSON listant les dépôts **amont** (upstream) consultés par un Virtual Repository.
- **priority** : ordre de résolution (plus grand = consulté en premier).
- Ici, on privilégie le dépôt privé (container-dev-java-repo) avant le cache de Maven Central.

7.2 Créer le Virtual Repository

```
gcloud artifacts repositories create virtual-maven-repo \
  --project=${PROJECT_ID} \
  --repository-format=maven \
  --mode=virtual-repository \
  --location=us-central1 \
  --description="Virtual Maven Repo" \
  --upstream-policy-file=./policy.json
```

- Crée un dépôt en mode **VIRTUAL_REPOSITORY**.
- Un dépôt virtual ne stocke pas (forcément) d'artefacts : il sert de **point d'entrée unique**.
- Il délègue la résolution aux dépôts upstream définis par **policy.json**.
- But : simplifier la configuration côté clients et réduire certains risques (ex. confusion de dépendances).

7.3 Générer la config Maven pour utiliser le Virtual Repository

```
gcloud artifacts print-settings mvn \
  --repository=virtual-maven-repo \
  --location=us-central1
```

- Génère la section Maven pour pointer vers **virtual-maven-repo**.
- Côté **pom.xml**, on remplace typiquement la section **<repositories>** par une seule entrée (le virtual).

7.4 Démonstration : réinitialiser le cache remote puis recompiler via le virtual

```
gcloud artifacts repositories delete maven-central-cache \
  --project=$PROJECT_ID \
  --location=us-central1 \
  --quiet
```

- Supprime le dépôt `maven-central-cache` sans demander de confirmation (`-quiet`).
- Objectif pédagogique : repartir d'un cache vide pour observer le remplissage.

```
gcloud artifacts repositories create maven-central-cache \
  --project=$PROJECT_ID \
  --repository-format=maven \
  --location=us-central1 \
  --description="Remote repository for Maven Central caching" \
  --mode=remote-repository \
  --remote-repo-config-desc="Maven Central" \
  --remote-mvn-repo=MAVEN-CENTRAL
```

- Recrée le dépôt remote (cache Maven Central) vide.

```
rm -rf ~/.m2/repository
mvn compile
```

- Vide le cache Maven local pour forcer des téléchargements.
- Compile le projet ; Maven pointe vers le **virtual**, qui redirige vers le dépôt privé ou le remote selon la politique.
- Effet observable : les artefacts finissent en cache dans `maven-central-cache` (car c'est lui qui proxy Maven Central), même si le client ne le référence pas directement.

8 Conclusion

Le lab met en place trois modes de dépôts Artifact Registry :

- **Standard** : stockage d'artefacts privés.
- **Remote** : proxy/cache d'un dépôt externe (Maven Central).
- **Virtual** : point d'accès unique, avec règles de priorité vers plusieurs upstream.